

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

DEPARTAMENTO DE INGENIERÍA EN SISTEMAS COMPUTACIONALES



Unidad de Aprendizaje:

Sistemas Operativos

Práctica

Inicio de sesión e interfaces basadas en texto

Equipo:
Alvarez Garcia Martin
Hernandez Castillo Emir Adrian
Joaquín Rojas González
Nava Alvarado Donovan Emilio

Grupo:
4CV4

Profesor:
José Alfredo Jiménez Benítez

Ciudad de México, 11 de octubre de 2026

Resumen

El presente proyecto consiste en el desarrollo de un sistema de inicio de sesión e interfaces basadas en texto, implementado en lenguaje C mediante la biblioteca `ncurses`. El sistema, denominado JDM E-TICKETS, permite realizar el registro de usuarios y la autenticación mediante una interfaz interactiva en terminal. Su diseño integra mecanismos de validación de datos, procesamiento criptográfico de contraseñas y almacenamiento local de información, con el propósito de establecer un flujo de acceso que contemple tanto la interacción con el usuario como el manejo de sus credenciales.

La interfaz permite desplazarse entre las opciones disponibles mediante el teclado, utilizando las teclas de dirección, `ENTER` y `ESC`. A través de los formularios se capturan los datos necesarios para crear una cuenta o iniciar sesión. Durante el registro se verifican condiciones como la longitud mínima del nombre de usuario, la estructura de la dirección de correo electrónico, la disponibilidad de ambos identificadores, la complejidad de la contraseña y la coincidencia entre la contraseña y su confirmación. Estas validaciones permiten controlar la información antes de completar el registro y proporcionan una respuesta cuando los datos ingresados no satisfacen los requisitos establecidos.

En materia de seguridad, el sistema utiliza la biblioteca `libsodium` para generar y verificar hashes de contraseñas mediante el algoritmo Argon2id. De esta manera, las contraseñas originales no necesitan almacenarse directamente en el archivo de usuarios. La información persistente se conserva localmente en `usuarios.txt`, donde cada registro contiene el nombre de usuario, el correo electrónico y el hash correspondiente. Este mecanismo permite mantener las cuentas disponibles entre distintas ejecuciones de la aplicación, sin depender de un sistema gestor de bases de datos o de un servicio remoto.

La organización del código fuente separa el flujo principal, las funciones de autenticación y los componentes de la interfaz, facilitando la identificación de las responsabilidades de cada módulo. Asimismo, la compilación se administra mediante un archivo `Makefile`, que define las opciones y las bibliotecas necesarias para generar el ejecutable.

El proyecto integra conceptos de programación en C, manejo de terminal, validación de entradas, persistencia en archivos y protección criptográfica de credenciales. Su implementación permite estudiar cómo interactúan estos elementos en una aplicación de autenticación de alcance local y cómo una interfaz textual puede proporcionar navegación, mensajes de estado y formularios estructurados sin recurrir a un entorno gráfico.

Palabras clave: sistemas operativos, lenguaje C, `ncurses`, autenticación, Argon2id, persistencia local y validación de datos.

Contenido

Resumen **1**

1. Introducción **3**

2. Marco Teórico **4**

2.1. Teoría sobre la creación de un inicio de sesión 4

2.2. Algoritmos de cifrado 4

2.3. Biblioteca ncurses 4

3. Desarrollo **5**

3.1. Organización del código fuente 5

3.2. Compilación del programa 5

3.3. Implementación de la interfaz con ncurses 6

3.4. Registro de usuarios y validación de datos 6

3.5. Cifrado de contraseñas y autenticación 7

3.6. Registro y persistencia en archivos 12

3.7. Pruebas funcionales 12

4. Conclusiones **14**

4.1. Alvarez Garcia Martin 14

4.2. Hernandez Castillo Emir Adrian 15

4.3. Joaquín Rojas González 16

4.4. Nava Alvarado Donovan Emilio 17

5. Referencias **18**

Anexos **19**

1 Introducción

Los sistemas de inicio de sesión constituyen un mecanismo fundamental para controlar el acceso a las aplicaciones que administran información asociada con distintos usuarios. Su funcionamiento requiere identificar a quien intenta ingresar, verificar sus credenciales y gestionar adecuadamente los datos utilizados durante el registro y la autenticación. Además de comprobar la correspondencia entre un nombre de usuario y una contraseña, resulta necesario contemplar la validación de las entradas, el almacenamiento persistente de las cuentas y la presentación de mensajes que permitan comprender el resultado de cada operación.

En el contexto de los sistemas operativos, el desarrollo de una aplicación de estas características permite estudiar la interacción entre un programa y el entorno en el que se ejecuta. El manejo del teclado, la presentación de información en una terminal, la lectura y escritura de archivos y el uso de bibliotecas externas forman parte de los elementos que intervienen en su funcionamiento. Estos componentes adquieren especial relevancia cuando la aplicación debe mantener una interacción organizada sin utilizar ventanas, botones gráficos ni otros elementos propios de una interfaz visual tradicional.

A partir de este contexto, se desarrolló JDM E-TICKETS, un sistema de registro e inicio de sesión implementado en lenguaje C. La aplicación utiliza la biblioteca `ncurses` para construir una interfaz basada en texto que permite navegar por las opciones y completar formularios mediante el teclado. Su funcionamiento contempla dos operaciones principales: la creación de cuentas nuevas y la autenticación de usuarios registrados. Para el registro, se establecen condiciones sobre el nombre de usuario, la dirección de correo electrónico y la contraseña, mientras que el inicio de sesión se basa en la existencia de la cuenta y la verificación de la contraseña proporcionada.

Otro aspecto relevante del proyecto es el tratamiento de las credenciales. En lugar de conservar las contraseñas originales, la aplicación emplea `libsodium` y el algoritmo Argon2id para generar hashes que posteriormente se utilizan durante la autenticación. Los registros se almacenan en un archivo local denominado `usuarios.txt`, lo que permite conservar la información entre ejecuciones sin incorporar una base de datos ni depender de un servidor externo. Esta decisión delimita el alcance de la solución y permite concentrar su implementación en el procesamiento de datos, la interacción con el usuario y las operaciones de archivos.

El objetivo principal del proyecto es implementar un mecanismo funcional de registro y autenticación que integre una interfaz textual, validaciones de entrada, verificación criptográfica de contraseñas y persistencia local. Para ello, el código se organiza en módulos que separan la lógica principal, las funciones de autenticación y los componentes visuales, mientras que el proceso de compilación se administra mediante un archivo `Makefile`.

El reporte presenta los fundamentos relacionados con el funcionamiento del sistema y describe las decisiones tomadas durante su implementación. Asimismo, aborda la estructura del código fuente, la compilación, la construcción de la interfaz, las reglas de validación, el tratamiento de las contraseñas y el almacenamiento de los usuarios. Finalmente, se incluyen las conclusiones correspondientes y las referencias utilizadas para contextualizar el desarrollo. De esta manera, se expone la relación entre los conceptos estudiados y su aplicación en un programa de autenticación ejecutado desde una terminal de Linux.

2 Marco Teórico

2.1 Teoría sobre la creación de un inicio de sesión

Un sistema de inicio de sesión (login) es el mecanismo de seguridad primario mediante el cual un sistema operativo o aplicación informática controla el acceso a sus recursos. Su función principal es autenticar la identidad de un usuario comparando las credenciales proporcionadas (generalmente un nombre de usuario y una contraseña) con los registros previamente almacenados de forma segura en el sistema.

2.2 Algoritmos de cifrado

El almacenamiento de contraseñas en texto plano representa una vulnerabilidad crítica. Para mitigar este riesgo, los sistemas implementan algoritmos de cifrado o funciones de un solo sentido (hashing). En lenguaje C, es común utilizar funciones matemáticas o bibliotecas nativas como `crypt()` (basada en DES, MD5 o SHA) que transforman la contraseña original en una cadena de caracteres ilegible. Durante el inicio de sesión, el sistema cifra la contraseña ingresada y compara el resultado con el hash guardado; si ambos coinciden, se otorga el acceso sin que el sistema necesite conocer la clave original.

2.3 Biblioteca ncurses

`ncurses` (new curses) es una biblioteca de programación que proporciona una API para construir interfaces de usuario basadas en texto de manera independiente de la terminal. Permite controlar la posición del cursor, crear ventanas, utilizar colores y leer el teclado carácter por carácter. Funciones como `noecho()` son esenciales en los inicios de sesión, ya que desactivan el eco visual de la terminal, permitiendo que el usuario escriba su contraseña sin que los caracteres sean visibles en la pantalla.

3 Desarrollo

El desarrollo del sistema de inicio de sesión se realizó en lenguaje C, utilizando la biblioteca `ncurses` para construir una interfaz de usuario basada en texto y la biblioteca `libsodium` para el procesamiento de contraseñas. El sistema permite registrar cuentas, autenticar usuarios y almacenar la información necesaria para conservar los registros entre ejecuciones.

La implementación se organizó en módulos para separar la lógica de autenticación, los componentes visuales y el flujo principal de ejecución. Esta distribución permite distinguir las responsabilidades de cada componente y facilita la comprensión de la estructura del programa.

3.1 Organización del código fuente

El proyecto se encuentra dividido en los archivos siguientes:

- `main.c`: contiene el flujo principal de ejecución y la coordinación de las operaciones de inicio de sesión y registro de usuarios.
- `auth.c`: implementa las funciones relacionadas con la autenticación, el registro de cuentas, la validación de credenciales y el manejo de la información persistente.
- `auth.h`: declara las funciones y las interfaces utilizadas por el módulo de autenticación.
- `ui.c`: contiene las funciones encargadas de mostrar los elementos de la interfaz mediante `ncurses` y procesar la interacción con el usuario.
- `ui.h`: declara las funciones y las interfaces correspondientes al módulo de presentación.
- `Makefile`: define las instrucciones necesarias para compilar y enlazar el programa.

La separación entre autenticación e interfaz permite mantener diferenciadas las operaciones que procesan las credenciales y aquellas que presentan los formularios y mensajes en pantalla. El archivo principal coordina ambos módulos para establecer el flujo de navegación de la aplicación.

3.2 Compilación del programa

La compilación se administra mediante un archivo `Makefile`, que reúne las opciones del compilador, los archivos fuente y las bibliotecas requeridas por el proyecto. Para generar el ejecutable se utiliza el compilador GCC con las opciones de lenguaje C11 y advertencias de compilación.

Entre las opciones de compilación se encuentran `-Wall`, `-Wextra` y `-Wpedantic`, que habilitan advertencias relacionadas con posibles errores y construcciones que no se ajustan al estándar seleccionado. La opción `-O2` aplica optimizaciones al código generado.

El programa requiere las bibliotecas `ncurses`, `tinfo` y `sodium`. Las bibliotecas de interfaz proporcionan las funciones necesarias para trabajar en la terminal, mientras que `libsodium` incorpora las operaciones criptográficas empleadas en el procesamiento de contraseñas.

La compilación puede realizarse desde el directorio del proyecto mediante el siguiente comando:

```
make
```

Cuando la compilación finaliza correctamente, se genera el ejecutable `acceso`. Para iniciar la aplicación se utiliza el siguiente comando:

```
./acceso
```

Este procedimiento permite generar y ejecutar el sistema desde los archivos fuente, siempre que las dependencias requeridas se encuentren instaladas en el entorno.

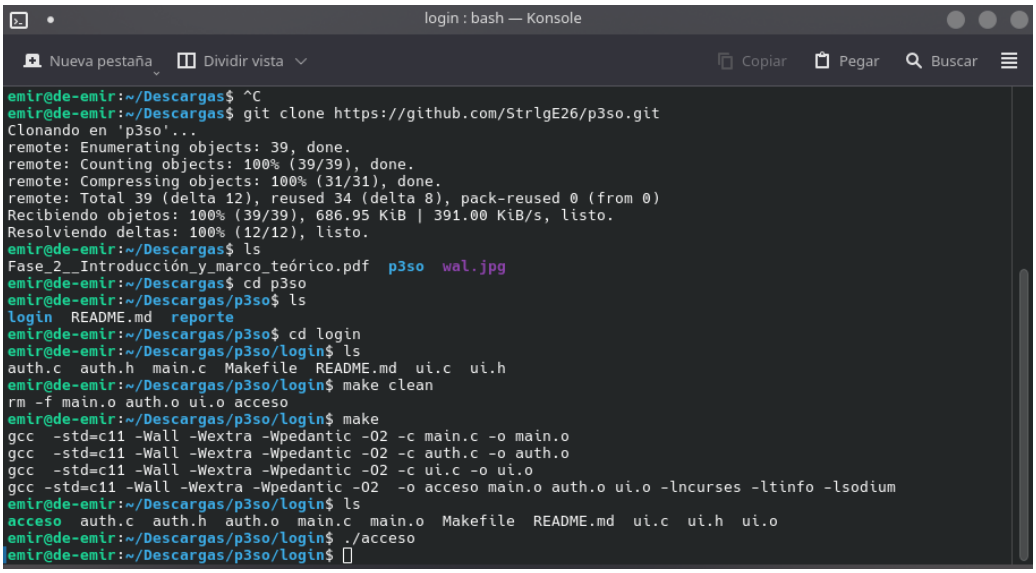


Figura 1: Compilación del sistema mediante el archivo `Makefile`.

3.3 Implementación de la interfaz con ncurses

La interfaz de usuario se desarrolló mediante la biblioteca `ncurses`, que permite controlar la presentación de texto, la posición del cursor y la lectura de entradas del teclado dentro de una terminal. Su utilización hace posible organizar los formularios y los mensajes de la aplicación sin recurrir a una interfaz gráfica tradicional.

La pantalla principal presenta las opciones relacionadas con el acceso y el registro de cuentas. Los campos de entrada se representan visualmente como cuadros de texto, mientras que las opciones seleccionadas se distinguen mediante resaltados de color. El diseño utiliza una combinación de fondo oscuro y texto claro para mantener una presentación uniforme.

La navegación se realiza mediante las teclas de dirección, la tecla `ENTER` y la tecla `ESC`. Las teclas de dirección permiten desplazarse entre las opciones disponibles, `ENTER` confirma la selección o permite continuar con una operación y `ESC` permite regresar o cancelar la acción correspondiente de acuerdo con el contexto de la interfaz.

La configuración de `ncurses` incluye funciones para controlar el eco de los caracteres, habilitar la lectura de teclas y procesar las teclas especiales. En particular, `noecho()` evita que la entrada se muestre automáticamente en pantalla, mientras que `keypad()` permite interpretar teclas especiales de acuerdo con la configuración del terminal.

El uso de estas funciones centraliza la interacción con el teclado y permite mantener un comportamiento consistente entre las pantallas de autenticación y registro.

3.4 Registro de usuarios y validación de datos

El registro de usuarios permite crear nuevas cuentas a partir de un nombre de usuario, una dirección de correo electrónico y una contraseña con su respectiva confirmación. Antes de almacenar una cuenta, el sistema

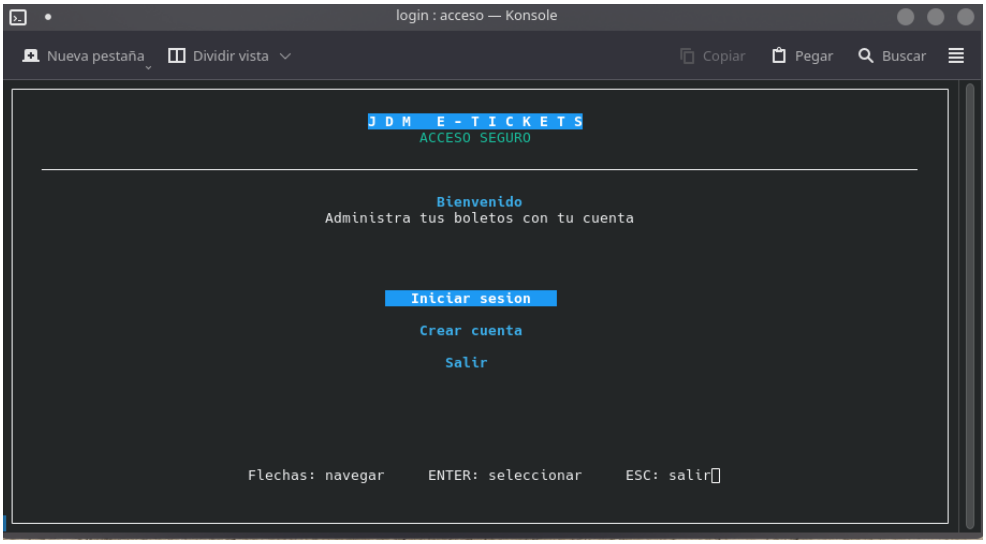


Figura 2: Menú principal de JDM E-TICKETS.

verifica que los datos ingresados cumplan con las reglas de validación definidas para el proyecto.

El nombre de usuario debe contener al menos cinco caracteres y no debe encontrarse registrado previamente. Esta condición evita la creación de cuentas con nombres demasiado cortos y permite mantener la unicidad del identificador utilizado para iniciar sesión.

La dirección de correo electrónico debe presentar una estructura válida y ser diferente de las direcciones que ya se encuentran registradas. La validación comprueba la sintaxis de la dirección proporcionada, pero no verifica que el buzón exista ni que el usuario tenga acceso a él.

La contraseña debe cumplir con los requisitos de complejidad establecidos, que incluyen letras mayúsculas, letras minúsculas, números y símbolos. Además, el sistema solicita una segunda entrada para confirmar la contraseña y ambas entradas deben coincidir para que el registro pueda continuar.

Cuando un campo no cumple con las reglas correspondientes, el flujo de registro permite corregir la entrada inválida sin perder los valores válidos que ya se habían introducido en los demás campos. De esta manera, la validación forma parte del proceso de captura de información y evita que una cuenta se registre con datos que incumplan las condiciones establecidas.

Estas comprobaciones se aplican durante el registro de usuarios. Su objetivo es reducir los errores de captura, evitar la duplicidad de identificadores y asegurar que la contraseña cumpla con los requisitos definidos antes de ser procesada para su almacenamiento.

3.5 Cifrado de contraseñas y autenticación

El tratamiento de las contraseñas constituye una parte esencial del sistema, ya que estos datos se utilizan para comprobar la identidad de los usuarios. Por esta razón, la aplicación emplea las funciones criptográficas de `libsodium` para generar y verificar hashes de contraseñas mediante el algoritmo Argon2id.

En lugar de guardar la contraseña original, el sistema utiliza la función `crypto_pwhash_str()` para generar una representación criptográfica destinada al almacenamiento. Este procedimiento incorpora los parámetros necesarios para que la contraseña pueda verificarse posteriormente sin conservar su valor en texto plano.

Durante el inicio de sesión, el sistema busca el registro correspondiente al nombre de usuario y utiliza la función `crypto_pwhash_str_verify()` para comprobar la contraseña ingresada frente al hash almacenado.

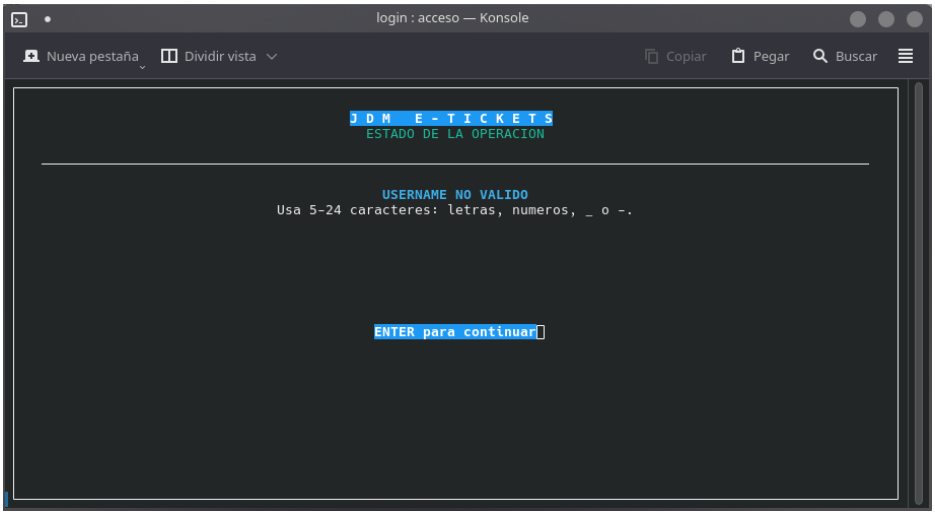


Figura 3: Validación de un nombre de usuario que no cumple con las condiciones de registro.

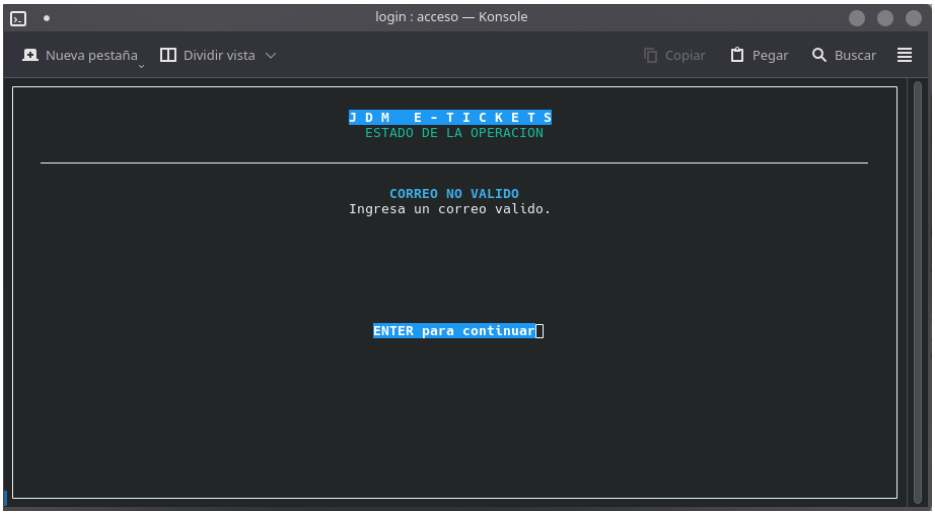


Figura 4: Mensaje de validación al ingresar una dirección de correo electrónico con estructura incorrecta.

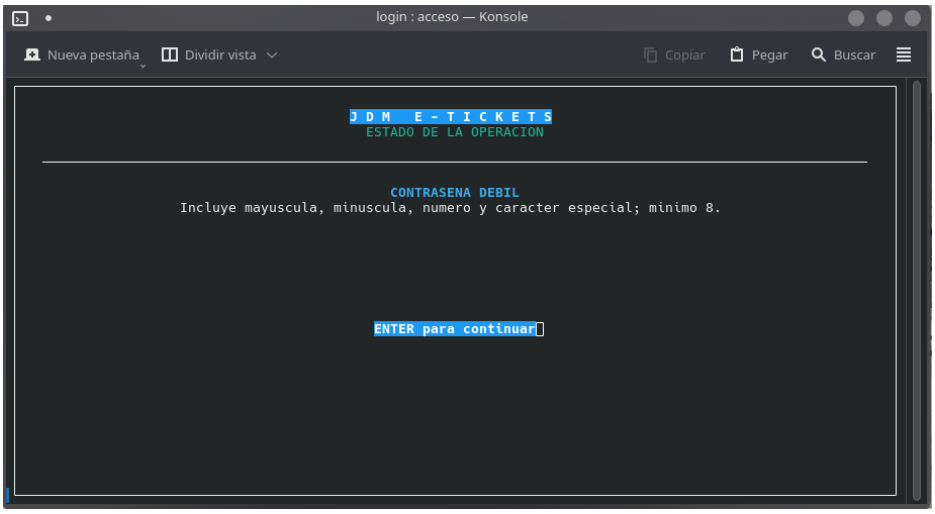


Figura 5: Aviso presentado cuando la contraseña no satisface los requisitos de complejidad.

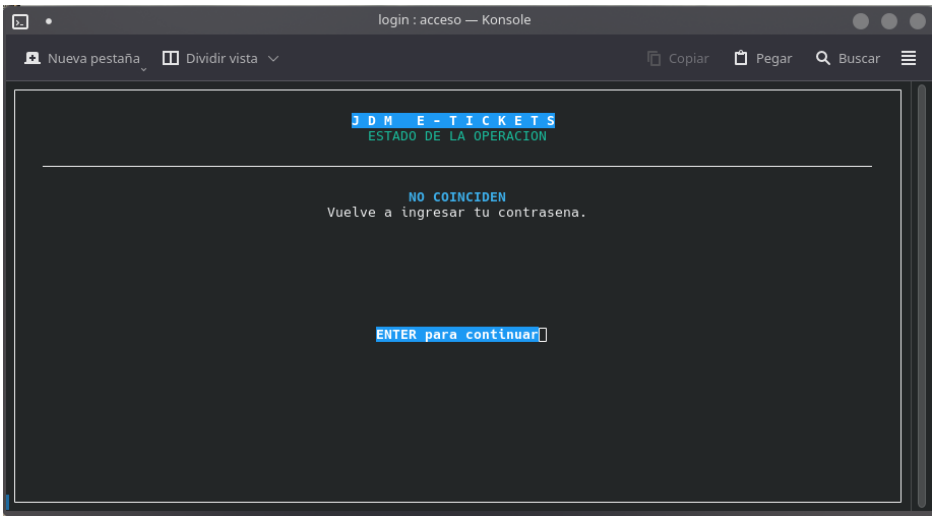


Figura 6: Mensaje presentado cuando la contraseña y su confirmación son diferentes.

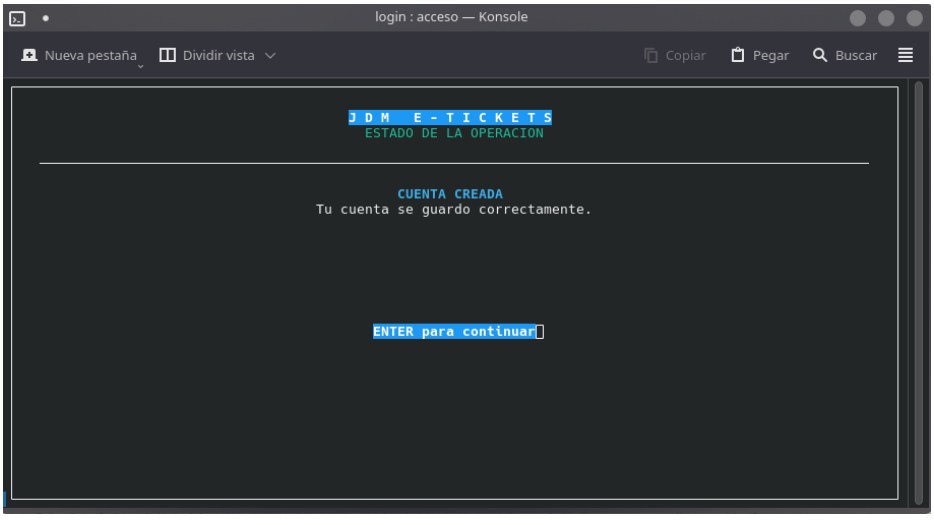


Figura 7: Confirmación visual del registro exitoso de una cuenta.

La autenticación se considera correcta cuando existe el usuario y la verificación criptográfica confirma que la contraseña proporcionada es válida.

El comportamiento de la interfaz depende del resultado de esta validación. Cuando la contraseña es incorrecta, el campo correspondiente se limpia para permitir una nueva entrada, mientras que el nombre de usuario se conserva. Si el nombre de usuario no existe, se limpian ambos campos, de modo que el siguiente intento comience con los datos de acceso vacíos.

El uso de Argon2id permite almacenar hashes diseñados específicamente para la verificación de contraseñas. Este mecanismo no equivale al cifrado reversible, ya que el sistema no necesita recuperar la contraseña original, sino comprobar si la contraseña proporcionada supera la verificación frente al hash registrado.

Adicionalmente, las funciones de limpieza de memoria de `libsodium`, como `sodium_memzero()`, permiten borrar explícitamente determinados datos sensibles de los búferes utilizados por el programa cuando corresponde. Esta medida complementa el manejo de credenciales al reducir el tiempo durante el cual ciertos datos permanecen en memoria.

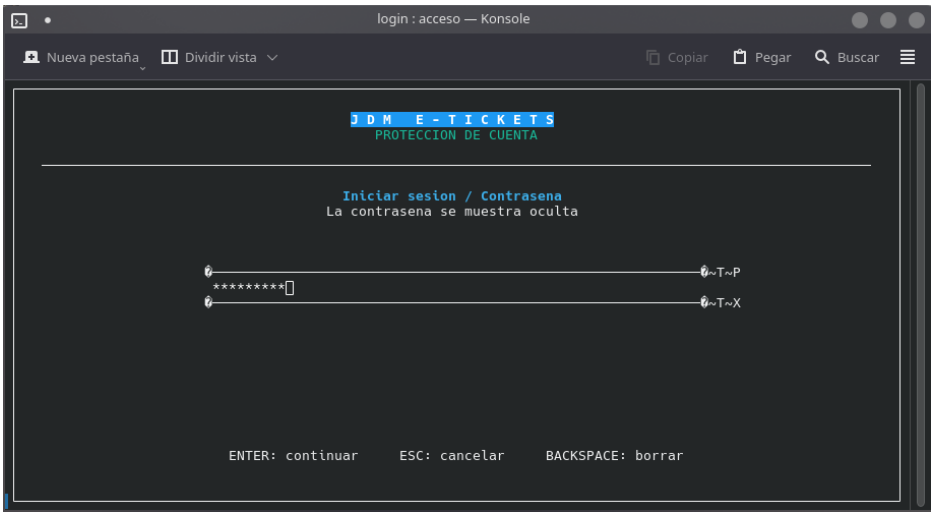


Figura 8: Formulario de inicio de sesión con el campo de contraseña.

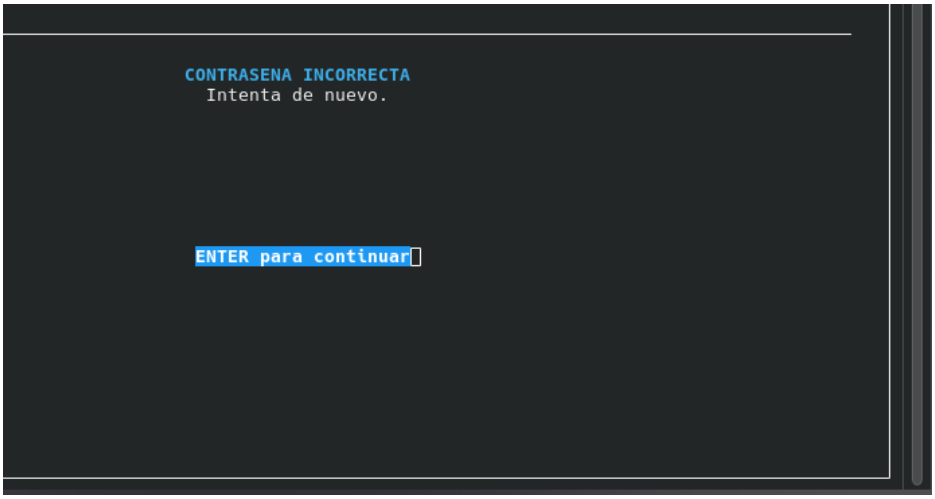


Figura 9: Respuesta de la aplicación ante una contraseña incorrecta.

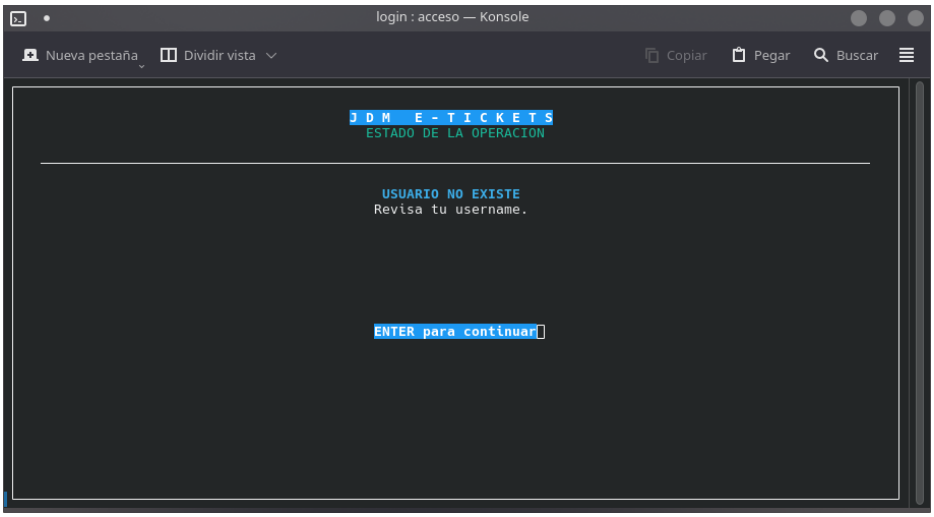


Figura 10: Respuesta de la aplicación cuando el nombre de usuario no está registrado.

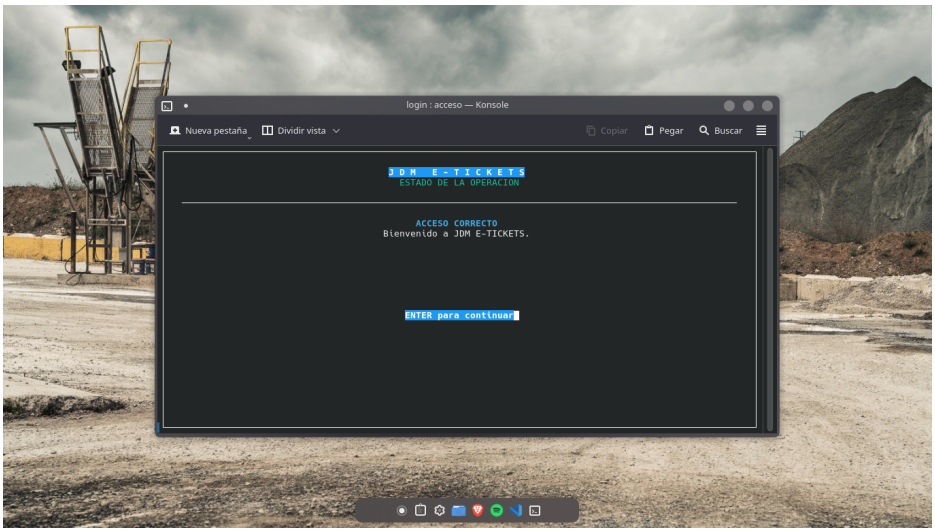


Figura 11: Confirmación visual de un inicio de sesión correcto.

3.6 Registro y persistencia en archivos

La información de las cuentas se conserva en un archivo local denominado `usuarios.txt`. Este mecanismo proporciona persistencia entre ejecuciones sin requerir un sistema gestor de bases de datos ni una conexión a un servidor remoto.

Los registros se almacenan utilizando una estructura delimitada por dos puntos, con los siguientes campos:

```
usuario:correo:hash
```

En esta estructura, el primer campo corresponde al nombre de usuario, el segundo contiene la dirección de correo electrónico y el tercero almacena el hash de la contraseña. La contraseña original no forma parte del registro persistente.

La lectura del archivo permite localizar las cuentas existentes y recuperar la información necesaria para comprobar la unicidad del nombre de usuario y del correo electrónico, así como para realizar la autenticación. El registro de una nueva cuenta incorpora los datos correspondientes al archivo para que permanezcan disponibles después de cerrar y volver a ejecutar el programa.

La persistencia local evita depender de servicios externos para las operaciones básicas del sistema. No obstante, la disponibilidad y la integridad de los registros dependen de los permisos, el estado y la administración del archivo en el equipo donde se ejecuta la aplicación.

El sistema utiliza un archivo de texto como mecanismo de almacenamiento, por lo que la protección de su contenido y el acceso al entorno de ejecución forman parte de las consideraciones de seguridad de la implementación. El almacenamiento de hashes, en lugar de contraseñas en texto plano, reduce la exposición directa de las credenciales, aunque no elimina la necesidad de proteger el archivo y el equipo.

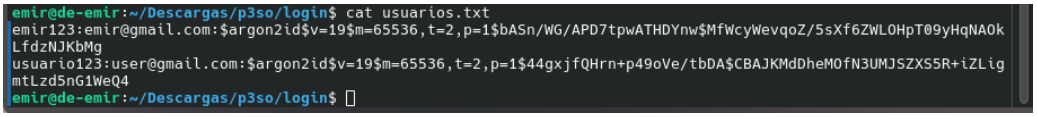


Figura 12: Estructura de los registros almacenados localmente en el archivo de usuarios.

3.7 Pruebas funcionales

Las pruebas funcionales permiten evaluar el comportamiento del sistema durante el registro de cuentas, la autenticación de usuarios y la conservación de los registros. Los escenarios considerados incluyen operaciones exitosas y situaciones en las que los datos ingresados no cumplen con las reglas de validación.

La Tabla 1 presenta los casos de prueba y el comportamiento esperado en cada escenario.

Tabla 1: Casos de prueba funcionales y resultados esperados.

N.º	Escenario	Entrada o condición	Resultado esperado
1	Registro correcto	Datos válidos y cuenta no registrada previamente	La cuenta se registra y queda almacenada para futuras ejecuciones.
2	Nombre de usuario corto	Nombre de usuario con menos de cinco caracteres	El registro no continúa hasta corregir el nombre de usuario.
3	Nombre de usuario duplicado	Nombre de usuario que ya existe en el archivo de usuarios	El sistema rechaza el registro duplicado y solicita una entrada válida.
4	Correo electrónico inválido	Dirección que no cumple con la estructura esperada	El sistema solicita corregir la dirección de correo electrónico.
5	Correo electrónico duplicado	Dirección que ya se encuentra registrada	El sistema rechaza el registro duplicado.
6	Contraseña débil	Contraseña que no incluye todos los tipos de caracteres requeridos	El sistema solicita una contraseña que cumpla las reglas de complejidad.
7	Confirmación incorrecta	Contraseña y confirmación diferentes	El registro no se completa hasta que ambas entradas coincidan.
8	Inicio de sesión correcto	Nombre de usuario existente y contraseña válida	El sistema permite completar la autenticación.
9	Contraseña incorrecta	Nombre de usuario existente y contraseña que no coincide	El sistema rechaza el acceso y limpia el campo de contraseña, conservando el nombre de usuario.
10	Usuario inexistente	Nombre de usuario que no está registrado	El sistema rechaza el acceso y limpia los campos de usuario y contraseña.
11	Persistencia de datos	Cuenta registrada y aplicación ejecutada nuevamente	La cuenta continúa disponible para las operaciones de autenticación.
12	Navegación por teclado	Uso de flechas, `ENTER` y `ESC`	La interfaz permite desplazarse, seleccionar opciones y regresar o cancelar según el contexto.

Los escenarios anteriores abarcan las operaciones principales del sistema y permiten distinguir los resultados de un registro válido de aquellos producidos por entradas incorrectas. La comprobación de estos comportamientos proporciona una referencia para evaluar la correspondencia entre las reglas de negocio, la lógica de autenticación y la interacción presentada en la terminal.

Los resultados observados durante las pruebas deben interpretarse en función de las condiciones de cada escenario. En particular, el registro correcto debe conservar la cuenta en el archivo local, mientras que los intentos con datos inválidos deben impedir la creación de registros que incumplan las reglas establecidas. De forma equivalente, el inicio de sesión debe permitir el acceso únicamente cuando la cuenta existe y la contraseña supera la verificación criptográfica.

La persistencia se evalúa mediante la disponibilidad de los registros después de finalizar y volver a iniciar la aplicación. Por otro lado, las pruebas de navegación permiten comprobar que los controles de teclado mantienen el comportamiento previsto en las distintas pantallas. En conjunto, estos escenarios describen los aspectos funcionales esenciales de la aplicación, sin sustituir las comprobaciones específicas de seguridad, concurrencia o resistencia ante modificaciones externas del archivo de usuarios.

4 Conclusiones

4.1 Alvarez Garcia Martin

4.2 Hernandez Castillo Emir Adrian

La realización de esta práctica me permitió comprender de manera más clara cómo se relacionan la programación en C, el manejo de archivos y la interacción con el usuario dentro de una aplicación ejecutada en Linux. Uno de los principales aprendizajes fue identificar que el desarrollo de un sistema de inicio de sesión no consiste únicamente en solicitar un nombre de usuario y una contraseña, sino que también requiere validar los datos, administrar las credenciales y mantener la información disponible entre distintas ejecuciones.

Uno de los aspectos que considero más relevantes fue la separación del código en módulos. Al organizar el flujo principal, la lógica de autenticación y los elementos de la interfaz en archivos independientes, comprendí la importancia de establecer responsabilidades específicas dentro del programa. Esta organización facilita la comprensión del funcionamiento del sistema y permite realizar modificaciones sin concentrar toda la lógica en un solo archivo.

El uso de `ncurses` también me permitió comprender mejor la interacción entre una aplicación y la terminal. Mediante el manejo de las teclas de dirección, `ENTER` y `ESC`, fue posible construir una interfaz que permite desplazarse entre las opciones y completar los formularios sin recurrir a ventanas gráficas. Además, el tratamiento de los mensajes de error y de los campos de entrada me ayudó a identificar la importancia de proporcionar respuestas claras cuando el usuario introduce información incorrecta.

En cuanto a la seguridad, comprendí que almacenar las contraseñas directamente en un archivo representa una práctica inadecuada para el manejo de credenciales. Por esta razón, el uso de `libsodium` y del algoritmo Argon2id permitió incorporar un mecanismo de generación y verificación de hashes sin necesidad de conservar las contraseñas originales. Este aspecto me permitió distinguir entre el cifrado reversible y el procesamiento de contraseñas mediante hashes, así como reconocer la importancia de utilizar funciones criptográficas especializadas en lugar de implementar estos procedimientos desde cero.

Por otra parte, el almacenamiento de las cuentas en el archivo `usuarios.txt` me permitió comprender que la persistencia de la información depende de las operaciones de lectura y escritura realizadas por el propio programa. Al no utilizar un gestor de bases de datos, el sistema debe encargarse de organizar los registros y recuperar los datos necesarios durante la autenticación. También identifiqué que mantener la información en un archivo local implica considerar su integridad, sus permisos y la protección de los datos almacenados.

Otro aprendizaje importante fue la necesidad de validar las entradas antes de completar el registro. Las condiciones relacionadas con el nombre de usuario, el correo electrónico, la complejidad de la contraseña y su confirmación permiten controlar los datos que recibe la aplicación y evitar registros que no cumplan con las reglas establecidas. Asimismo, el tratamiento diferenciado de los errores de autenticación demuestra que el comportamiento de la interfaz debe corresponder con el resultado de las comprobaciones realizadas por el programa.

Finalmente, el uso de un archivo `Makefile` me permitió reconocer la importancia de organizar el proceso de compilación y especificar las dependencias necesarias para generar el ejecutable. En conclusión, esta práctica me permitió relacionar los conceptos de programación en C, manejo de archivos, entrada y salida mediante terminal y protección de credenciales con su aplicación en un sistema funcional. Considero que el principal aprendizaje fue comprender que la confiabilidad de una aplicación de autenticación depende de la integración adecuada de sus componentes y no solamente de que las operaciones principales puedan ejecutarse. Estos conocimientos constituyen una base para continuar desarrollando aplicaciones en Linux con una estructura más organizada y un manejo más cuidadoso de la información.

4.3 Joaquín Rojas González

4.4 Nava Alvarado Donovan Emilio

5 Referencias

- Kerrisk, M. (2010). *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press.
- Stallman, R., & McGrath, R. (2004). *GNU Make: A Program for Directing Recompilation*. Free Software Foundation.
- Strang, J. (1986). *Programming with curses*. O'Reilly Media.

Anexos

En esta sección se incluye el repositorio oficial donde se encuentra el código fuente en lenguaje C, los archivos de cabecera y el archivo Makefile necesarios para la compilación del sistema:

`https://github.com/StrlgE26/p3so.git`